

Master Control Plan

MCP · Version 1.0 · Confidential

Document Type

Master Control Plan — Project Execution Strategy

Project

FinNewt.bi — Personal Finance Intelligence Platform

Version

1.0

Methodology

Agile + Modular Microservice Architecture

Development Phases

9 Phases (UI/UX → Deployment)

Prepared By

Engineering Lead — 30+ Years SDE

Date

2025

Table of Contents

Section	Title	Page
1	Project Execution Strategy	3
2	Development Phases Overview	3
3	Phase 1 — UI/UX Design	4
4	Phase 2 — Frontend Development	4
5	Phase 3 — Backend Development	5
6	Phase 4 — Database & Cache	6
7	Phase 5 — Analytics Engine	6
8	Phase 6 — Reporting System	7
9	Phase 7 — Security Hardening	7
10	Phase 8 — Testing Strategy	8
11	Phase 9 — Deployment	9
12	Project Structure	10
13	Team Role Breakdown	10
14	Risk Management	11
15	Success Metrics	12
16	Final Vision	12

1. Project Execution Strategy

FinNewt.bi is executed under an Agile + Modular Architecture methodology. This hybrid approach provides the iterative velocity of Agile sprints while enforcing the structural discipline of a well-defined modular architecture. Each module is designed for independent development, testing, and deployment, enabling parallel workstreams across engineering teams.

◆ Methodology Rationale

Principle	Rationale	Benefit
Agile Sprints (2-week)	Iterative delivery with frequent stakeholder feedback	Reduced risk of scope creep
Modular Architecture	Each service/module is independently deployable	Isolated failures, easier debugging
Domain-Driven Design	Services aligned with business domains	Clear ownership and boundaries
API-First Development	Contracts defined before implementation	Parallel frontend/backend development
Continuous Integration	Automated build/test on every commit	Early defect detection
Infrastructure as Code	Deployment configs version-controlled	Reproducible environments

2. Development Phases Overview

Phase	Name	Duration (Est.)	Key Output
1	UI/UX Design	2 Weeks	Design system, component library, Figma wireframes
2	Frontend Development	4 Weeks	Functional React SPA, API-ready components
3	Backend Development	5 Weeks	All microservices with RESTful APIs
4	Database & Cache	1 Week	Production-ready Supabase schema + Redis config
5	Analytics Engine	2 Weeks	Rule-based insight engine, trend algorithms
6	Reporting System	1 Week	CSV / PDF export pipeline
7	Security Hardening	1 Week	Penetration testing, auth hardening, audit
8	Testing	2 Weeks	Unit, integration, E2E, and security test suites
9	Deployment	1 Week	Production-ready CI/CD pipeline, live environment

Total Estimated Timeline: 19 Weeks (approximately 5 months) for v1.0 production launch. Timeline assumes a 3–4 person core engineering team with dedicated QA and DevOps support in Phases 8–9.

3. Phase 1 — UI/UX Design

PHASE 1 — UI/UX DESIGN

Objective: Establish the complete visual design system and component architecture before any frontend code is written.

Key Tasks:

- Design and prototype Landing Page with hero section, feature highlights, and CTA
- Define dashboard layout: sidebar navigation, header, main content grid
- Build atomic component library: buttons, inputs, cards, modals, badges, charts
- Design all page templates: Auth, Dashboard, Transactions, Analytics, Goals, Settings
- Implement responsive breakpoint system (mobile-first, 5 breakpoints)
- Define animation choreography: page transitions, micro-interactions, loading states
- Establish Tailwind design tokens: color palette, spacing scale, typography scale
- Conduct accessibility audit on design system (WCAG 2.1 AA compliance)

Deliverables:

- Complete Figma/design specification document with all screens
- React component architecture document mapping UI to component hierarchy
- Tailwind configuration file with custom design tokens
- Storybook or equivalent component documentation

4. Phase 2 — Frontend Development

PHASE 2 — FRONTEND DEVELOPMENT

Objective: Implement the full React TypeScript SPA with all pages, components, state management, and API integration layer.

Key Tasks:

- Scaffold project with Vite + React + TypeScript + TailwindCSS
- Configure routing with React Router v6 (protected and public routes)
- Implement Authentication pages: Login, Register, OTP Verify, Forgot Password
- Build Dashboard page with KPI cards, chart components, insights panel, recent transactions
- Build Transactions page: list view, filters, create/edit/delete modals
- Build Budget page: category budgets, progress bars, alert indicators
- Build Analytics page: Recharts integration (pie, line, bar, heatmap charts)
- Build Goals page: goal cards, progress rings, contribution form
- Build Notifications page: inbox, read/unread states
- Build Reports page: export triggers, preview panel
- Build Settings page: profile form, notification preferences, theme toggle
- Implement Zustand or Redux Toolkit global state management
- Create API service layer with Axios interceptors for JWT attachment and refresh
- Implement Framer Motion animations across all page transitions and interactions
- Configure dark/light mode with system preference detection

Deliverables:

- Fully functional frontend SPA with all pages and components
- API-ready service layer awaiting backend endpoint completion
- Responsive design verified across desktop, tablet, and mobile viewports
- TypeScript type definitions for all data models matching API contracts

5. Phase 3 — Backend Development

PHASE 3 — BACKEND DEVELOPMENT

Objective: Implement all backend microservices: Auth Service and Finance Service in Java Spring Boot; Analytics and Notification Services in Node.js.

Key Tasks:

- Scaffold API Gateway with routing, CORS, rate limiting, and JWT validation middleware
- Auth Service: implement /register, /verify-otp, /login, /forgot-password, /refresh endpoints
- Auth Service: BCrypt password hashing, JWT token factory, Redis OTP storage with TTL
- Finance Service: Transaction CRUD with pagination, filtering, and category logic
- Finance Service: Budget CRUD with real-time utilization calculation
- Finance Service: Goals CRUD with progress and projection computation
- Analytics Service (Node.js): trend aggregation queries, rule engine executor
- Notification Service (Node.js): SMTP mailer, scheduled job runner (node-cron)
- Notification Service: Budget alert triggers, goal reminder events, weekly digest
- Implement shared DTOs and error response envelope across all services
- Configure inter-service communication patterns for cross-domain events
- Set up structured logging (SLF4J / Winston) and centralized error handling

Deliverables:

- All API endpoints documented and functional per API Design specification
- Authentication service with OTP email verification fully operational
- Finance, Analytics, and Notification services passing integration tests
- API Gateway operational with routing, auth, and rate limiting enforced

◆ Service Technology Summary

Service	Technology	Port	Responsibilities
API Gateway	Spring Cloud Gateway / Nginx	8080	Routing, Auth validation, Rate limiting, CORS
Auth Service	Java Spring Boot	8081	Registration, OTP, JWT issuance, Password reset
Finance Service	Java Spring Boot	8082	Transactions, Budgets, Goals CRUD operations
Analytics Service	Node.js + Express	8083	Insights engine, Trend aggregation, Summaries
Notification Service	Node.js + Express	8084	SMTP email, Scheduled jobs, Alert dispatching

6. Phase 4 — Database & Cache

PHASE 4 — DATABASE & CACHE SETUP

Objective: Provision and configure the Supabase PostgreSQL database and Redis cache layer for all services.

Key Tasks:

- Provision Supabase project and configure connection pooling (PgBouncer)
- Execute schema migrations for all tables: users, transactions, budgets, goals, notifications
- Configure Row-Level Security (RLS) policies to enforce per-user data isolation
- Create database indexes for high-frequency query patterns (user_id, date, category)
- Set up Redis instance with appropriate eviction policies and persistence config
- Implement Redis OTP cache: key-value with TTL expiry and atomic increment for retry limits
- Implement Redis dashboard cache with configurable TTL (default: 5 minutes)
- Implement Redis rate limiting with sliding window counters per IP/user
- Configure database backup schedule (daily automated snapshots, 30-day retention)
- Document all schema migrations and seed scripts for reproducible environments

Deliverables:

- Production-ready database with all tables, RLS policies, and indexes
- Redis configured for OTP, session, rate limiting, and dashboard caching
- Migration scripts version-controlled and executable in CI/CD pipeline
- Database connection monitoring and slow query logging enabled

7. Phase 5 — Analytics Engine

PHASE 5 — ANALYTICS ENGINE

Objective: Build the intelligent core of FinNewt.bi — the rule-based behavioral insight engine and trend processing pipeline.

Key Tasks:

- Implement spending trend aggregation: daily, weekly, monthly, yearly breakdowns
- Build category-level spend analysis with month-over-month delta computation
- Implement budget compliance scoring (% utilized per category per period)
- Build the rule execution engine: iterate rules against user transaction aggregates
- Implement all INS-001 through INS-007 insight rules (per SRS Section 8)
- Build savings velocity calculator for goal completion projection
- Implement recurring transaction pattern detector using frequency analysis
- Build spending heatmap data aggregator (spend by day-of-week and hour-of-day)
- Optimize all analytics queries with database indexes and Redis result caching
- Create rule configuration system to allow future rules without code changes

Deliverables:

- Fully operational analytics pipeline producing insights for any user data set
- All 7 insight rules implemented, tested, and producing meaningful recommendations
- Analytics API endpoints returning sub-500ms responses with caching
- Insight rule configuration documentation for future extensibility

8. Phase 6 — Reporting System

PHASE 6 — REPORTING SYSTEM

Objective: Implement the financial report generation pipeline supporting both CSV and branded PDF exports.

Key Tasks:

- Implement CSV export: configurable date range, column selection, UTF-8 encoding
- Implement PDF report generation using a server-side rendering pipeline
- Embed charts (pie, line graphs) as static SVG/PNG within PDF reports
- Design branded PDF report template with FinNewt.bi header, colors, and typography
- Implement report summary section: income/expense totals, savings rate, top categories
- Add transaction table with pagination across multiple PDF pages if needed
- Implement async report generation with email delivery for large datasets
- Store generated reports temporarily in object storage for download link delivery

Deliverables:

- CSV export fully functional with filtering and column selection
- Branded PDF reports generating correctly with embedded charts
- Report generation API endpoints tested and operational
- Large dataset handling via async generation + email delivery

9. Phase 7 — Security Hardening

PHASE 7 — SECURITY HARDENING

Objective: Conduct a systematic security review and harden all layers of the application against common attack vectors.

Key Tasks:

- Audit all API endpoints for missing authentication and authorization checks
- Verify JWT validation on every protected route through API Gateway
- Test OTP rate limiting: confirm lockout after 3 failed attempts
- Conduct OWASP Top 10 vulnerability assessment across all services
- Verify all database queries use parameterized statements (no raw SQL interpolation)
- Test CORS policy: confirm cross-origin requests are blocked from non-whitelisted origins
- Validate input sanitization on all user-controlled fields
- Verify BCrypt cost factor and ensure no plaintext credentials in logs
- Conduct dependency vulnerability scan using npm audit and OWASP Dependency Check
- Implement security response headers: CSP, X-Frame-Options, X-Content-Type-Options
- Penetration test authentication flows: brute force, session fixation, token replay
- Document all findings and remediation actions in security audit report

Deliverables:

- Security audit report with all findings documented and remediated
- OWASP Top 10 assessment completed with no critical or high-severity findings
- Dependency vulnerability scan showing no known critical CVEs
- Security hardening checklist signed off by Engineering Lead

10. Phase 8 — Testing Strategy

A multi-layer testing strategy ensures that every component, integration point, and user workflow meets quality and reliability standards before production deployment.

Test Type	Scope	Tools	Coverage Target
Unit Testing	Individual functions, service methods, utility logic	JUnit 5 (Java), Jest (Node.js)	80% line coverage
API Testing	All REST endpoint contracts, response codes, payloads	Postman, REST Assured	100% critical endpoints
Integration Testing	Service-to-service communication, DB interactions	Spring Boot Test, Supertest	All service boundaries
End-to-End (E2E) Testing	Critical user journeys: Register→Login→Transfer→Report	Playwright, Cypress	5 core user journeys
Security Testing	Auth flows, input validation, rate limiting	OWASP ZAP, Burp Suite	OWASP Top 10
Performance Testing	API response times under simulated load	Apache JMeter	< 500ms at 100 concurrent
Regression Testing	Full test suite run on every CI/CD push	GitHub Actions pipeline	100% automated on CI
Accessibility Testing	WCAG 2.1 AA compliance on frontend	axe-core, Lighthouse	Zero critical violations

◆ Defect Management Process

Severity	Definition	Resolution SLA
P0 — Critical	Production blocker, data loss, security breach	Immediate — same day
P1 — High	Core feature broken, no workaround available	24 hours
P2 — Medium	Feature impaired, workaround exists	3 business days
P3 — Low	Minor UX issue, cosmetic defect	Sprint backlog

11. Phase 9 — Deployment

PHASE 9 — DEPLOYMENT & LAUNCH

Objective: Configure and execute the full production deployment pipeline for all services and infrastructure components.

Key Tasks:

- Containerize all services with Docker (multi-stage builds for optimized image sizes)
- Write Docker Compose configuration for local development environment
- Configure GitHub Actions CI/CD pipeline: lint → test → build → deploy
- Deploy frontend to Vercel/Netlify with environment-based configuration
- Deploy backend services to Render/Railway with auto-deploy on main branch
- Configure environment variables and secrets via platform secret manager
- Set up custom domain with SSL certificate (Let's Encrypt / Cloudflare)
- Configure production Supabase instance with connection pooling
- Set up uptime monitoring (Better Uptime / UptimeRobot) with alert escalation
- Configure error tracking with Sentry for both frontend and backend services
- Execute production smoke test across all critical user flows
- Document runbook: deployment procedure, rollback procedure, incident response

Deliverables:

- All services live in production with CI/CD pipeline operational
- Custom domain configured with valid SSL certificate
- Monitoring, alerting, and error tracking fully operational
- Deployment runbook and incident response playbook documented

◆ Deployment Architecture

Component	Platform	Configuration Notes
Frontend (React SPA)	Vercel / Netlify	CDN-distributed, auto-preview on PRs
API Gateway	Render / Railway	Autoscaling, health check endpoint
Auth Service	Render / Railway	Environment secrets, Redis connection
Finance Service	Render / Railway	DB connection pooling via PgBouncer
Analytics Service	Render / Railway	Stateless, horizontal scaling
Notification Service	Render / Railway	Scheduled jobs, SMTP credentials
PostgreSQL	Supabase	Managed, daily backups, RLS enabled
Redis	Render / Redis Cloud	Persistent, eviction policy: allkeys-lru
CI/CD	GitHub Actions	Lint → Test → Build → Deploy on push to main
Monitoring	Better Uptime + Sentry	99% uptime target, P0 alert < 5 min

12. Project Structure

12.1 Frontend Directory Structure

```
frontend/
  ■■■ public/ ← Static assets, favicon
  ■■■ src/
    ■ ■■■ components/ ← Atomic & molecular UI components
    ■ ■ ■■■ ui/ ← Base components (Button, Input, Card)
    ■ ■ ■■■ charts/ ← Recharts wrapper components
    ■ ■ ■■■ layout/ ← Sidebar, Header, Footer
    ■ ■■■ pages/ ← Route-level page components
    ■ ■■■ hooks/ ← Custom React hooks
    ■ ■■■ store/ ← Zustand / Redux state slices
    ■ ■■■ services/ ← Axios API service modules
    ■ ■■■ types/ ← TypeScript interfaces & enums
    ■ ■■■ utils/ ← Helper functions, formatters
    ■ ■■■ routes/ ← Route definitions, guards
```

12.2 Backend Directory Structure

```
backend/
  ■■■ api-gateway/ ← Routing, JWT validation, rate limiting
  ■■■ auth-service/ ← Registration, OTP, JWT (Java Spring Boot)
  ■ ■■■ src/main/java/
    ■ ■ ■■■ controller/
    ■ ■ ■■■ service/
    ■ ■ ■■■ repository/
    ■ ■ ■■■ config/
  ■■■ finance-service/ ← Transactions, Budgets, Goals (Java)
  ■■■ analytics-service/ ← Insights engine, trends (Node.js)
  ■■■ notification-service/ ← SMTP, scheduled jobs (Node.js)
  ■■■ shared/ ← Common DTOs, error models, constants
```

13. Team Role Breakdown

Role	Phases Active	Core Responsibilities
Engineering Lead / Architect	All phases	Architecture decisions, code review, technical direction, risk management
Frontend Developer	1, 2, 8	React SPA, component library, state management, API integration
Backend Developer (Java)	3, 7, 8	Auth & Finance microservices, API design, database layer
Backend Developer (Node.js)	3, 5, 6, 8	Analytics engine, Notification service, report generation
Database Engineer	4, 7	Schema design, indexing strategy, RLS policies, query optimization
QA Engineer	8, 7	Test plan authoring, manual and automated testing, defect tracking
DevOps Engineer	9, 4	CI/CD pipeline, Docker containerization, cloud deployment, monitoring
UX Designer	1	Wireframes, design system, accessibility review, prototype validation

14. Risk Management

The following risk register identifies key threats to project delivery and defines mitigation strategies to maintain schedule, quality, and security targets.

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
R-01	OTP delivery failures (SMTP outages)	Medium	High	Retry logic with backoff; fallback SMTP provider (SendGrid → AWS SES)
R-02	Slow analytics query performance	High	Medium	Redis query result caching; compound DB indexes; denormalization
R-03	API endpoint overload under high traffic	Medium	High	Redis rate limiting; horizontal scaling; API Gateway circuit breaker
R-04	Data inconsistency across microservices	Medium	High	Database transactions for multi-step operations; idempotent APIs
R-05	Security vulnerabilities in auth flow	Low	Critical	Security hardening phase; OWASP audit; penetration testing; bug bounty
R-06	Frontend performance on low-end devices	Medium	Medium	Code splitting, lazy loading, virtual lists for large datasets
R-07	Third-party service downtime (Supabase, Redis, Algolia)	High	High	Health checks; fallback handling; multi-region database replication
R-08	Scope creep from uncontrolled feature additions	High	Medium	Strict change control; v1.0 scope freeze; future enhancements backlog
R-09	Key team member unavailability	Medium	High	Cross-training; documented architecture; code review culture
R-10	JWT token theft or replay attacks	Low	Critical	HTTP-only cookies; short access token TTL; refresh token rotation

15. Success Metrics

15.1 Technical Metrics

Metric	Target	Measurement Method
API P95 Response Time	< 500ms	APM tool (Datadog / Render metrics)
Dashboard Load Time	< 2 seconds	Lighthouse CI on every deployment
Platform Monthly Uptime	≥ 99.0%	Uptime monitoring with monthly reports
Redis Cache Hit Rate	≥ 80%	Redis INFO stats dashboard
Unit Test Coverage	≥ 80%	Jest / JaCoCo coverage reports in CI
Critical Bug Count at Launch	0 P0/P1 bugs	QA sign-off gate before deployment
Security Vulnerabilities	0 Critical / High CVEs	OWASP Dependency Check in CI

15.2 Product Metrics

Metric	Target	Rationale
Insight Accuracy Rate	> 90% user-validated relevance	Core product value proposition
UI Responsiveness	Verified across 5 device sizes	Broad accessibility requirement
Onboarding Completion	> 80% of new users complete setup	UX simplicity validation
Transaction Entry Speed	< 30 seconds for a standard entry	Workflow efficiency target
Report Generation Time	< 10 seconds for 12-month dataset	User experience benchmark

16. Final Vision

FinNewt.bi v1.0 establishes a solid, production-grade foundation — but the long-term vision is far more ambitious. The product roadmap envisions an evolution from a capable personal finance tracker into a fully autonomous intelligent financial operating system.

◆ **Product Evolution Roadmap**

Horizon	Phase	Capability
Now (v1.0)	Foundation	Expense tracking, budgets, goals, rule-based insights, reports
Near (v2.0)	Intelligence	AI financial assistant (LLM), OCR receipt scanning, subscription intelligence
Mid (v3.0)	Integration	Bank Open Banking sync, UPI real-time integration, voice logging
Far (v4.0)	Autonomy	ML predictive analytics, automated savings rules, investment tracking
Vision (v5.0+)	Platform	Multi-user family wallets, fintech SaaS platform, API marketplace

Architectural Note from Engineering Lead: Every architectural decision made in FinNewt.bi v1.0 has been deliberately forward-compatible with this roadmap. The microservice separation, stateless APIs, event-driven notification pipeline, and modular analytics engine are all designed to accommodate AI/ML integration, third-party financial data connectors, and multi-tenancy without requiring foundational rewrites. Build it right once.

FinNewt.bi — Transforming personal finance through intelligence, discipline, and beautiful engineering.